Elements of AIML Lab
Experiment 08 (After-submission Reference): PCA (Dimensionality Reduction)

Tofik Ali

March 1, 2026

Contents

1 How to use this reference **1**

2 PCA: what it learns **1**

3 Worked example (end-to-end) **2**
3.1 Step 1: load data and select features 2
3.2 Step 2: scale + fit PCA 2
3.3 Step 3: pick number of components 2
3.4 Step 4: plot cumulative explained variance 2
3.5 Step 5: interpret components (loadings) 3

4 Leakage-safe PCA (important) **3**

5 Troubleshooting **3**

1 How to use this reference

This reference is meant to be shared **after you submit** Experiment 08. It clarifies:

- why scaling is needed before PCA,
- how to choose number of components using explained variance,
- how to avoid leakage when PCA is used before a model.

2 PCA: what it learns

PCA finds new axes (principal components) such that:

- PC1 captures maximum variance,
- PC2 captures maximum remaining variance subject to being orthogonal to PC1,
- and so on.

Each principal component is a weighted combination of original features.

3 Worked example (end-to-end)

3.1 Step 1: load data and select features

```
import pandas as pd

df = pd.read_csv("processed.csv") # or your dataset
X = df.select_dtypes("number") # PCA requires numeric features
```

If you have a label column, keep it separately (do not include in PCA features).

3.2 Step 2: scale + fit PCA

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA()
Z = pca.fit_transform(X_scaled)

evr = pca.explained_variance_ratio_
cum = np.cumsum(evr)
print("First 10 EVR:", evr[:10])
print("First 10 cumulative:", cum[:10])
```

3.3 Step 3: pick number of components

Two common choices:

- choose the smallest k such that cumulative EVR ≥ 0.90 (90% variance),
- or choose k at the "elbow" of the cumulative curve (diminishing returns).

You can also let scikit-learn choose by ratio:

```
pca90 = PCA(n_components=0.90)
Z90 = pca90.fit_transform(X_scaled)
print("Chosen components:", pca90.n_components_)
```

3.4 Step 4: plot cumulative explained variance

```
import matplotlib.pyplot as plt

plt.plot(cum, marker="o")
plt.axhline(0.90, color="red", linestyle="--", label="90%")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.grid(True)
plt.legend()
plt.show()
```

3.5 Step 5: interpret components (loadings)

Loadings show which original features contribute to each PC.

```
import pandas as pd

loadings = pd.DataFrame(
    pca.components_.T,
    index=X.columns,
    columns=[f"PC{i+1}" for i in range(pca.n_components_)]
)

print(loadings["PC1"].sort_values(key=abs, ascending=False).head(8))
```

Interpretation tip: large magnitude loading ⇒ that feature strongly influences that component.

4 Leakage-safe PCA (important)

If you use PCA before training a model:

- split train/test first,
- fit scaler+PCA on training only,
- transform test using fitted objects,
- ideally use a pipeline.

Example:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.90)),
    ("clf", LogisticRegression(max_iter=1000)),
])
```

5 Troubleshooting

- **PCA fails:** check for non-numeric columns and missing values.
- **EVR is low:** your features may be weakly correlated; PCA may not compress much.
- **PC plot looks noisy:** PCA is not a classifier; separation is not guaranteed.

Sources: Provost and Fawcett, *Data Science for Business.*;McKinney, *Python for Data Analysis.*;WEKA Documentation (Waikato Environment for Knowledge Analysis), accessed 2026-02-28.;Matplotlib and Seaborn documentation, accessed 2026-02-28.